Let $\mathcal{Z}$ be the latent space and $\mathcal{X}$ be the data space. We take a known prior distribution over $\mathcal{Z}$

$$z \sim p_z(z).$$

We have a (generator) function $G : \mathcal{Z} \to \mathcal{X}$

$$x = G(z; \theta).$$

This generator induces a distribution over the data space $\mathcal{X}$,

$$p_{model} = p_G(x) = (G)_{\#pz} = p_z(G^{-1}) \left|\det[\mathbf{J}_{G^{-1}}(x)]\right|.$$

The above explicit formula is defined if and only if the generator G is invertible, and is derived from transformation formula. See `[Transformation of Random Variables]` for detail.

In practice, G is rarely invertible so we can only induce an implicit distribution, in a sense that we cannot get the explicit formula for its density but we can sample from it.

To compare the implicit distribution p_{model} and the true data distribution p_{data}, we introduce a (discriminator) function $D : \mathcal{X} \to [0, 1]$ and a functional V that depends on p_{data}, p_{model} and D.

0.0.1 Optimal discriminator theory

Let $p(x)$ and $q(x)$ be two arbitrary probability densities over $\mathcal{X}$. We define

$$V(p, q, D) = \int_{\mathcal{X}} \left(p(x) \log D(x) + q(x) \log\left(1 - D(x)\right)\right) dx$$

equivalent to

$$V(p, q, D) = \mathbb{E}_{x\sim p}\left[\log D(x)\right] + \mathbb{E}_{x\sim q}\left[\log\left(1 - D(x)\right)\right]$$

We can see from the results of analysis that the function $D^*(x)$ which maximizes this function is

$$D^*(x) = \frac{p(x)}{p(x) + q(x)}$$

0.0.2 GAN objective

In the context of GANs, we set $p = p_{data}$ and $q = p_{model}$. The discriminator D is trained to maximize $V(G, D)$ for a fixed G.

$$\max_D V(G, D) = \mathbb{E}_{x\sim p_{data}}\left[\log D(x)\right] + \mathbb{E}_{x\sim p_{model}}\left[\log(1 - D(x))\right]$$

From the previous section result, the optimal discriminator is

$$D^*_G(G) = \frac{p_{data}(x)}{p_{data}(x) + p_{model}(x)}$$

The generator's cost function $C(G)$ is defined as the value of this maximized functional

$$C(G) = \max_D V(G, D) = V(G, D^*_G)$$

Substituting $D_G^*(x)$ into expression of $C(G)$,

$$\begin{aligned} C(G) &= \mathbb{E}_{x\sim p_{data}}\left[\log\left(\frac{p_{data}(x)}{p_{data}(x)+p_{model}(x)}\right)\right] + \mathbb{E}_{x\sim p_{model}}\left[\log\left(1-\frac{p_{data}(x)}{p_{data}(x)+p_{model}(x)}\right)\right] \\ &= \mathbb{E}_{x\sim p_{data}}\left[\log\left(\frac{p_{data}(x)}{p_{data}(x)+p_{model}(x)}\right)\right] + \mathbb{E}_{x\sim p_{model}}\left[\log\left(\frac{p_{model}(x)}{p_{data}(x)+p_{model}(x)}\right)\right] \end{aligned}$$

This expression can be shown to be related to the Jensen-Shannon Divergence (JSD). Let $M = \frac{p_{data}+p_{model}}{2}$. Then

$$\begin{aligned} C(G) &= \mathbb{E}_{x\sim p_{data}}\left[\log\left(\frac{p_{data}(x)}{M(x)}\cdot\frac{1}{2}\right)\right] + \mathbb{E}_{x\sim p_{model}}\left[\log\left(\frac{p_{model}(x)}{M(x)}\cdot\frac{1}{2}\right)\right] \\ &= \mathbb{E}_{x\sim p_{data}}\left[\log\left(\frac{p_{data}}{M}\right)\right] - \log 2 + \mathbb{E}_{x\sim p_{model}}\left[\log\left(\frac{p_{model}}{M}\right)\right] - \log 2 \\ &= D_{KL}(p_{data}\|M) + D_{KL}(p_{model}\|M) - 2\log 2 \\ &= 2\cdot D_{JS}(p_{data}\|p_{model}) - 2\log 2 \end{aligned}$$

Therefore, the generator's objective in the $\min_G \max_D V(G,D)$ game

$$\min_G C(G) = \min_G \left[2\cdot D_{JS}(p_{data}\|p_{model}) - 2\log 2\right]$$

is equivalent to minimizing the Jensen-Shannon divergence between the real data distribution and the generator's distribution.
The global minimum is achieved when $p_{model} = p_{data}$, where $D_{JS} = 0$ and $D^*(x) = 1/2$.

$D(x)$ represents the discriminator's estimate of the probability that an input x came from the real data distribution p_{data} (as opposed to the generator's distribution p_G).

0.0.3 GAN objective in practice

While the theoretical value of the game is $C(G) = \min_G \max_D V(G,D)$, this is not the objective the generator directly optimizes in practice.

In the original paper, the generator's objective is defined as minimizing the value of $V(G,D)$:

$$\min_G V(G,D) = \min_G \left(\mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1-D(G(z)))]\right)$$

Since the first term $\mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)]$ does not depend on G, this is equivalent to:

$$\min_G \mathbb{E}_{z\sim p_z}[\log(1-D(G(z)))]$$

Problem: This loss function "saturates." When the discriminator is very confident (i.e., $D(G(z))$ is close to 0, meaning it easily identifies fakes), the gradient of $\log(1-D(G(z)))$ becomes very small (it's flat). This provides almost no gradient for the generator to learn from, especially early in training.

To fix this, the authors proposed an alternative objective for the generator. Instead of minimizing the probability of being caught, the generator *maximizes* the probability of being right:

$$\max_G \mathbb{E}_{z\sim p_z}[\log D(G(z))]$$

Benefit: This objective has much stronger gradients early in training when the generator is poor (when $D(G(z))$ is close to 0). It provides a much more stable training dynamic.

Both generator objectives (min-max and non-saturating) have the same fixed point, but the non-saturating version provides better gradients and is almost universally used in practice.

0.0.4 Training algorithm of generator

Algorithm 1 Minibatch stochastic gradient descent training of Generative Adversarial Networks

Input: Discriminator steps k, minibatch size m
Initialize generator parameters θ_g and discriminator parameters θ_d
for number of training iterations **do**
  for k steps **do**
    Sample minibatch $\{x^{(1)}, \ldots, x^{(m)}\} \sim p_{\text{data}}(x)$
    Sample minibatch $\{z^{(1)}, \ldots, z^{(m)}\} \sim p_z(z)$
    Update the discriminator by ascending its stochastic gradient: $\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \Big[\log D(x^{(i)}) + \log\left(1 - D(G(z^{(i)}))\right) \Big]$
  end for
  Sample minibatch $\{z^{(1)}, \ldots, z^{(m)}\} \sim p_z(z)$
  Update the generator by descending its stochastic gradient (non-saturating loss):$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log D(G(z^{(i)}))$
end for

0.1 References

Generative Adversarial Nets, Goodfellow et al. (2014)
NIPS 2016 Tutorial: Generative Adversarial Networks, Goodfellow. (2016)